Prática 3 Blockchain: Da Teoria à Prática

Aluno(a): Bianca Guimarães & Eduardo Pereira.

ADS Turma 039, manhã, 5 Período.

Link do Colab:
https://colab.research.google.com/drive/10QCwgMIGZeExtzRDmzITcVhsexa8xfKw

Rodei o notebook inteiro que o professor passou, com as 5 práticas sobre blockchain. Fui fazendo os desafios de cada uma e pesquisando o que não lembrava direito, pra entender de verdade o que estava rolando em cada bloco de código.

Prática 1 : Blockchain do zero em Python
Nessa parte criei as classes Block e Blockchain, que geram uma cadeia onde cada bloco guarda o hash do bloco anterior. Testei criando 4 blocos e a validação deu True (cadeia íntegra).

Desafio 1: tentei alterar o bloco 1, mudando os dados de uma transação da Bob para uma transação fake da "Mallory" com valor 9999. Antes disso a validação era True, depois da alteração virou False. Isso acontece porque o hash de cada bloco é calculado em cima do próprio conteúdo dele (index, timestamp, data, hash anterior, nonce) então só de mudar o "data", o hash calculado já fica diferente do hash que tinha sido salvo ali. E como cada bloco também guarda o hash do bloco anterior, mexer em um bloco no meio da cadeia quebra a ligação com todos os blocos seguintes. É basicamente essa a lógica que dá segurança pra blockchain: para "consertar" uma fraude, teria que recalcular tudo de novo a partir daquele ponto.

Prática 2 : Prova de Trabalho (Proof of Work)
Aqui implementei a mineração: o código fica testando valores de "nonce" até achar um hash que comece com N zeros (a dificuldade). Com dificuldade 4, o notebook achou o hash em 258.308 tentativas, levando 0,83 segundos.

Desafio 2: testei a mineração com dificuldade 3, 4 e 5, e o tempo mudou muito:
Dificuldade 3 → 1.434 tentativas, tempo quase 0.
Dificuldade 4 → 35.830 tentativas, 0,11s.
Dificuldade 5 → 1.038.587 tentativas, 3,30s.

Pesquisando um pouco entendi que cada zero a mais multiplica o espaço de busca por 16 (porque o hash é hexadecimal), então o tempo cresce super rápido (exponencial). É exatamente por causa disso que fica caro demais tentar reescrever blocos antigos de uma blockchain de verdade e também é o motivo de todo mundo falar tanto do gasto de energia do Bitcoin, porque é literalmente processamento bruto tentando "adivinhar" o nonce certo.

Prática 3 : Assinaturas digitais e carteiras
Usei a biblioteca ecdsa pra gerar um par de chaves (privada e pública) pra Alice e pra Bob,

no mesmo padrão usado por Bitcoin e Ethereum (curva secp256k1). Alice assinou uma transação e Bob conseguiu verificar a assinatura dela (deu True).

Desafio 3: testei duas situações de ataque. Primeiro, se a Mallory tentasse verificar a assinatura usando a própria chave pública dela (fingindo ser a Alice) deu False, não verificou. Segundo, se alguém mudasse um único caractere da mensagem original (troquei o valor "100" por "999") a verificação também deu False. Isso faz sentido porque a assinatura depende matematicamente tanto de quem assinou quanto do conteúdo exato da mensagem (ela passa por um hash SHA-256 antes de ser assinada). Ou seja, ninguém consegue forjar uma assinatura em nome de outra pessoa, nem alterar uma transação já assinada sem que isso seja detectado.

Prática 4: Smart Contracts (simulado)
Simulei um token no estilo ERC-20 chamado AulaToken, com 1.000.000 de moedas, todas começando na carteira da Alice. Fiz uma transferência de 250 tokens da Alice pra Bob, e funcionou (Alice ficou com 999.750, Bob com 250).

Desafio 4: tentei fazer o Bob transferir mais tokens do que ele realmente tinha (pedi pra transferir 999.999, mas ele só tinha 250). O contrato recusou a operação (retornou False) e o saldo do Bob continuou igual. Isso simula o comportamento do require do Solidity, uma trava que impede a função de continuar se a condição não for satisfeita. É esse tipo de regra que garante que um smart contract nunca deixa um saldo ficar negativo, sem precisar de nenhum "banco" verificando por fora.

Prática 5: Web3: conexão com blockchain real
Essa parte usa a biblioteca web3.py pra tentar ler dados reais direto da rede Ethereum. Como não criei uma conta na Alchemy/Infura pra pegar uma API key (precisaria de cadastro), o notebook rodou no "modo demo", com dados fictícios: mostrou o saldo do endereço público do Vitalik Buterin como 1.2345 ETH (valor fake) e um número de bloco fictício (#19000000).

Desafio 5: não cheguei a consultar dados reais porque isso dependia de criar a conta na Alchemy, mas o modo demo já deixa claro como a chamada funcionaria numa conexão de verdade só trocando a variável ALCHEMY_URL por uma API key real, o mesmo código passaria a trazer o saldo, o bloco e as transações reais da mainnet.

No fim das contas, a prática que mais me deu esclarecimentos foi a da Prova de Trabalh, só de ver o tempo subindo absurdamente de uma dificuldade pra outra dá pra entender na prática porque mineração de bitcoin gasta tanta energia. E a assinatura digital também ajudou a entender como uma transação "prova" que foi realmente eu que autorizei ela, sem precisar de nenhum banco no meio.